PROJECT REPORT

EXPERIMENT - 13

Project Statement

Design and develop a Smart Inventory Management System for Construction Sites. The system tracks materials and inventory movement to improve resource utilization and site management efficiency.

Submitted by

Abinaya S – 24MER001

Dhiyanasree KK – 24MER008

Puthiyaraj M – 24MER045

Rithick S – 24MER051

Smart Inventory Management System for Construction Sites

Project Overview

The Smart Inventory Management System for Construction Sites is an IoT-based system designed to track and monitor construction materials using RFID technology. Construction sites handle a large number of materials such as tools, equipment, electrical components, pipes, fasteners, and other resources. Manual inventory management can result in material loss, incorrect records, and difficulty in tracking material movement.

This proposed system uses an ESP8266 Node MCU as the main controller and an MFRC522 RFID reader to identify RFID-tagged materials. Each material or material category can be associated with a unique RFID tag/card. When a tag is scanned, the system identifies the corresponding item and records its movement. A buzzer provides an audible indication whenever an RFID tag is detected.

The system provides a simple prototype for improving inventory visibility, reducing manual errors, and supporting efficient material utilization at construction sites.

Main Objective:

- To automate identification of construction materials using RFID technology.
- To track the movement of tagged materials.
- To reduce manual inventory recording errors.
- To improve material utilization and availability monitoring.
- To provide immediate identification when an RFID tag is scanned.
- To develop a low-cost IoT-based inventory monitoring prototype.
- To provide a foundation for future cloud-based inventory tracking.

Problem Statement

Construction sites handle a large number of materials, tools, and equipment, making inventory management difficult and time-consuming when performed manually. Improper tracking can lead to material loss, misplacement, inaccurate inventory records, delays, and inefficient resource utilization. Therefore, there is a need for a smart and automated system that can identify construction materials and monitor their movement in real time.

The proposed Smart Inventory Management System uses an ESP8266 Node MCU and MFRC522 RFID reader to identify RFID-tagged materials, provide alerts through a buzzer, and support efficient tracking of inventory movement. The system aims to reduce manual errors and improve overall inventory management at construction sites.

Problems Addressed:

Traditional construction-site inventory management can face several problems:

- Manual entry of material information is time-consuming.
- Materials may be misplaced or difficult to locate.
- Inventory records may not always be updated accurately.
- Material movement between storage and work areas can be difficult to track.
- Unauthorized or unexpected material movement may go unnoticed.
- Poor inventory visibility can lead to unnecessary material purchases.

The proposed RFID-based system addresses these issues by providing automated material identification and movement tracking.

Proposed Solution

The proposed system attaches or associates RFID tags/cards with construction materials or inventory categories. An MFRC522 RFID reader is connected to the ESP8266 Node MCU. Whenever a tag is brought close to the RFID reader, the reader detects its unique identification number and sends the information to the Node MCU.

The Node MCU processes the RFID information and determines the corresponding inventory item. A buzzer is activated to indicate successful detection. The system can be extended to store the scanned item ID, movement status, and timestamp in a cloud database.

Proposed Operation:

The system operates through the following sequence:

- The ESP8266 Node MCU is powered on.
- The MFRC522 RFID reader is initialized.
- RFID tags are assigned to different construction materials.
- When a tagged material approaches the RFID reader, the RFID reader detects the tag.
- The unique RFID identification number is transferred to the Node MCU.
- The Node MCU compares the detected ID with the predefined inventory information.
- The corresponding material is identified.
- The buzzer provides an audible indication of successful scanning.
- The inventory movement can be recorded for monitoring purposes.
- The process continues for subsequent RFID scans.

System Architecture

The system consists of five major sections:

Input Section:

The MFRC522 RFID Reader acts as the primary input device. RFID tags/cards attached to or associated with construction materials provide unique identification information.

Processing Section:

The ESP8266 Node MCU receives the RFID information, processes the tag ID, identifies the corresponding inventory item, and determines the required inventory status.

Alert Section:

The Buzzer provides an audible indication when an RFID tag is successfully detected. It can also be programmed to provide different alerts for valid and invalid tags.

Inventory Section:

The RFID tags/cards represent individual construction materials or inventory categories. Each tag has a unique identification number that can be mapped to a particular material.

Communication Section:

The ESP8266 provides Wi-Fi connectivity. In an expanded version, the detected RFID information can be transmitted to a cloud platform or web dashboard for remote inventory monitoring.

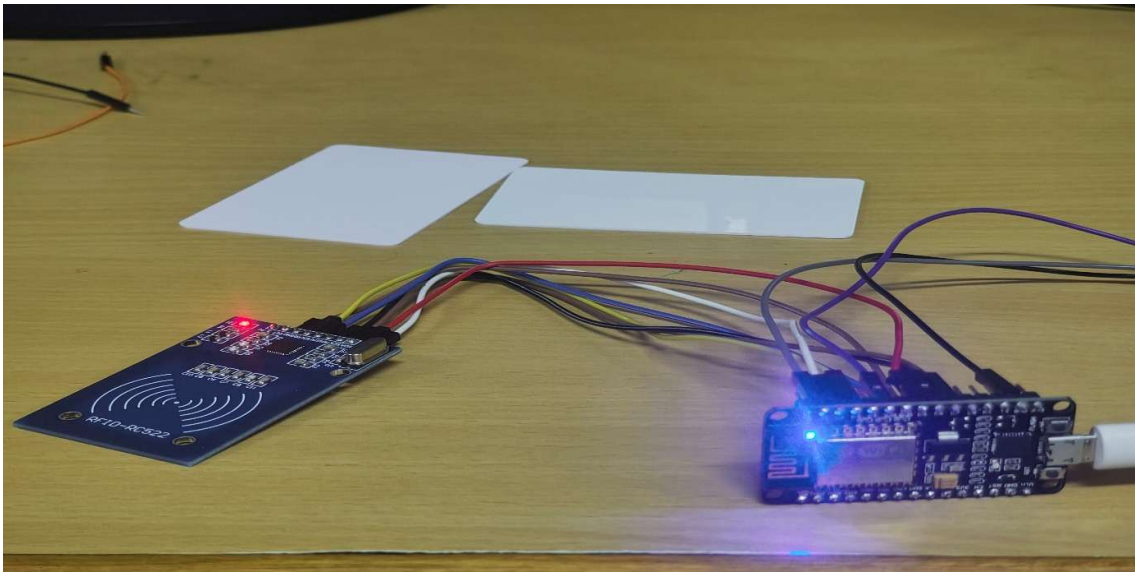
User Section:

Site supervisors, storekeepers, or inventory managers can use the collected information to monitor material movement, identify inventory items, and improve resource management.

Block Diagram:

RFID Tag/Card
↓
MFRC522 RFID Reader
↓
ESP8266 Node MCU
↓
RFID ID Processing & Inventory Identification
↓
Buzzer Alert
↓
Inventory Movement Record
↓
Wi-Fi / Cloud Dashboard (Future Enhancement)

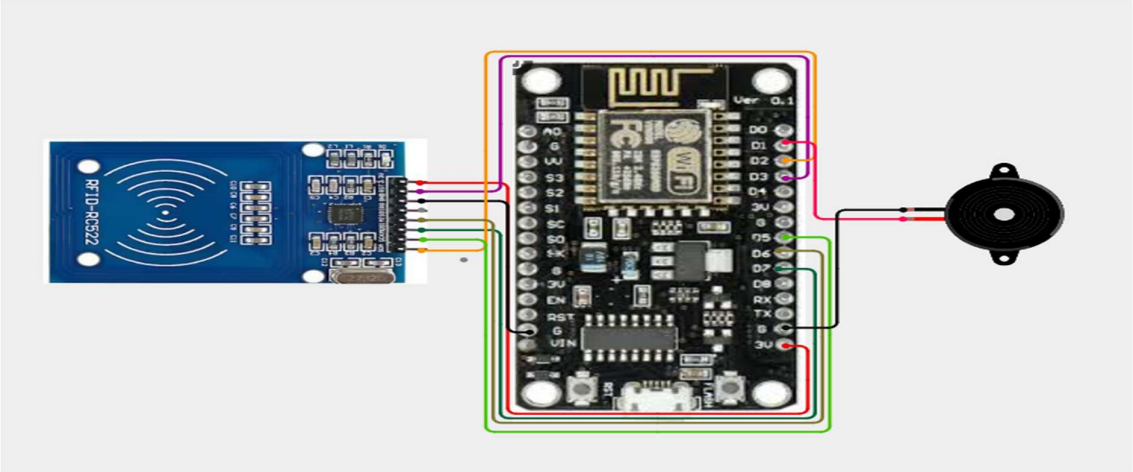
Image:



Circuit Table

S. No	Component Pin	ESP8266 Node MCU Pin	Function
1	MFRC522 VCC	3.3V	Power supply
2	MFRC522 GND	GND	Ground connection
3	MFRC522 SDA/SS	D2 (GPIO4)	SPI Chip Select
4	MFRC522 SCK	D5 (GPIO14)	SPI Clock
5	MFRC522 MOSI	D7 (GPIO13)	SPI Data Output
6	MFRC522 MISO	D6 (GPIO12)	SPI Data Input
7	MFRC522 RST	D1 (GPIO5)	RFID Reset
8	Buzzer (+)	D3 (GPIO0)	Buzzer control
9	Buzzer (−)	GND	Ground connection
10	RFID Tags/Cards	MFRC522 Reader	Material identification

Circuit Design



Important Pin Mapping:

ESP8266 → MFRC522

- D1 → RST
- D2 → SDA/SS
- D5 → SCK
- D6 → MISO
- D7 → MOSI
- 3.3V → VCC
- GND → GND

ESP8266 → Buzzer

- D3 → Buzzer (+)
- GND → Buzzer (−)

Algorithm

- Start the System.
- Initialize the ESP8266 Node MCU.
- Initialize SPI communication between the ESP8266 and MFRC522 RFID reader.
- Initialize the buzzer as an output device.
- Register the RFID tag IDs with their corresponding construction materials.
- Continuously check whether an RFID tag/card is present near the reader.
- If no tag is detected, continue scanning.
- If a tag is detected, read its unique RFID ID.
- Compare the detected RFID ID with the registered inventory IDs.
- If the ID is valid, identify the corresponding construction material.
- Activate the buzzer to indicate successful identification.
- Record the material movement or update its inventory status.
- If the RFID ID is not registered, generate an invalid-tag indication.
- Wait for the next RFID tag/card.
- Repeat the scanning and tracking process continuously.

- Stop the system when power is switched off.

Coding

```
#include <ESP8266WiFi.h>
#include <SPI.h>
#include <MFRC522.h>
#include <ThingSpeak.h>

// ========================================================
// WiFi
// ========================================================
const char* WIFI_SSID = "Rithick";
const char* WIFI_PASSWORD = "12345678";

// ========================================================
// ThingSpeak
// ========================================================
unsigned long CHANNEL_ID = 3462882;
const char* WRITE_API_KEY = "L0OMJAXR2W8YBN2A";

WiFiClient client;

// ========================================================
// RFID MFRC522
// ========================================================
```

```
#define SS_PIN  4
#define RST_PIN 3

MFRC522 rfid(SS_PIN, RST_PIN);

// ============================================================
// Buzzer
// ============================================================
#define BUZZER_PIN 2

// ============================================================
// Inventory
// ============================================================
int cementStock = 0;
int steelStock = 0;

// ============================================================
// RFID cards
// First card  = Cement
// Second card = Steel
// ============================================================
unsigned long cementUID = 0;
unsigned long steelUID = 0;

bool cementRegistered = false;
bool steelRegistered = false;
```

```
// ============================================================
// Prevent repeated scanning
// ============================================================
unsigned long lastScanTime = 0;
const unsigned long scanDelay = 3000;

// ============================================================
// Setup
// ============================================================
void setup()
{
  Serial.begin(115200);
  delay(1000);

  Serial.println();
  Serial.println("==========================================");
  Serial.println(" SMART CONSTRUCTION INVENTORY SYSTEM");
  Serial.println("==========================================");

  // Buzzer
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  // RFID
  SPI.begin();
```

```
  rfid.PCD_Init();

  Serial.println("RFID reader initialized.");

  // WiFi
  connectWiFi();

  // ThingSpeak
  ThingSpeak.begin(client);

  Serial.println();
  Serial.println("System Ready.");
  Serial.println();

  Serial.println("Scan the FIRST RFID card for CEMENT.");
  Serial.println("Scan the SECOND RFID card for STEEL.");
  Serial.println();
}

// ============================================================
// Main Loop
// ============================================================
void loop()
{
  // Check WiFi
  if (WiFi.status() != WL_CONNECTED)
```

```
{
  connectWiFi();
}

// Check for RFID card
if (!rfid.PICC_IsNewCardPresent())
{
  return;
}

if (!rfid.PICC_ReadCardSerial())
{
  return;
}

// Prevent repeated scans
if (millis() - lastScanTime < scanDelay)
{
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
  return;
}

lastScanTime = millis();

// ================================================
```

```
// Read RFID UID
// ========================================================
unsigned long currentUID = 0;

Serial.println("---------------------------------------");
Serial.print("RFID UID: ");

for (byte i = 0; i < rfid.uid.size; i++)
{
  Serial.print(rfid.uid.uidByte[i], HEX);
  Serial.print(" ");

  currentUID = (currentUID << 8) | rfid.uid.uidByte[i];
}

Serial.println();

Serial.print("UID Number: ");
Serial.println(currentUID);

// ========================================================
// Register first RFID as Cement
// ========================================================
if (!cementRegistered)
{
  cementUID = currentUID;
```

```
  cementRegistered = true;

  Serial.println();
  Serial.println("FIRST CARD REGISTERED");
  Serial.println("Material: CEMENT");
  Serial.println("This card is now assigned to Cement.");

  beepSuccess();

  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();

  return;
}

// ======================================================
// Register second RFID as Steel
// ======================================================
if (!steelRegistered && currentUID != cementUID)
{
  steelUID = currentUID;
  steelRegistered = true;

  Serial.println();
  Serial.println("SECOND CARD REGISTERED");
  Serial.println("Material: STEEL");
```

```
  Serial.println("This card is now assigned to Steel.");

  beepSuccess();

  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();

  return;
}

// ========================================================
// Check Cement card
// ========================================================
if (currentUID == cementUID)
{
  handleCement();
}

// ========================================================
// Check Steel card
// ========================================================
else if (currentUID == steelUID)
{
  handleSteel();
}
```

```
  // ================================================
  // Unknown card
  // ================================================
  else
  {
    Serial.println();
    Serial.println("UNKNOWN RFID CARD!");
    Serial.println("Card is not registered.");

    beepError();
  }

  // Stop RFID communication
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();

  Serial.println("--------------------------------------");
  Serial.println();
}

// ================================================
// Cement Handling
// ================================================
void handleCement()
{
  Serial.println();
```

```
Serial.println("MATERIAL: CEMENT");

// Alternate between IN and OUT
if (cementStock == 0)
{
  // First scan = IN
  cementStock++;

  Serial.println("STATUS: IN");
  Serial.println("Cement added to inventory.");
}
else
{
  // Next scan = OUT
  cementStock--;

  Serial.println("STATUS: OUT");
  Serial.println("Cement removed from inventory.");
}

Serial.print("Cement Stock: ");
Serial.println(cementStock);

beepSuccess();

// Send to ThingSpeak
```

```
  sendToThingSpeak("Cement", "IN/OUT");
}

// ========================================================
// Steel Handling
// ========================================================
void handleSteel()
{
  Serial.println();
  Serial.println("MATERIAL: STEEL");

  // Alternate between IN and OUT
  if (steelStock == 0)
  {
    // First scan = IN
    steelStock++;

    Serial.println("STATUS: IN");
    Serial.println("Steel added to inventory.");
  }
  else
  {
    // Next scan = OUT
    steelStock--;

    Serial.println("STATUS: OUT");
```

```
    Serial.println("Steel removed from inventory.");
  }

  Serial.print("Steel Stock: ");
  Serial.println(steelStock);

  beepSuccess();

  // Send to ThingSpeak
  sendToThingSpeak("Steel", "IN/OUT");
}

// =======================================================
// Send Data to ThingSpeak
// =======================================================
void sendToThingSpeak(String material, String status)
{
  Serial.println();
  Serial.println("Sending data to ThingSpeak...");

  // Field 1 = Cement stock
  ThingSpeak.setField(1, cementStock);

  // Field 2 = Steel stock
  ThingSpeak.setField(2, steelStock);
```

```
// Field 3 = Last scanned material
if (material == "Cement")
{
  ThingSpeak.setField(3, 1);
}
else
{
  ThingSpeak.setField(3, 2);
}

// Field 4 = RFID UID
// Explicit cast fixes the ambiguous setField error
unsigned long uidToSend;

if (material == "Cement")
{
  uidToSend = cementUID;
}
else
{
  uidToSend = steelUID;
}

ThingSpeak.setField(4, (long)uidToSend);

// Field 5 = IN/OUT status
```

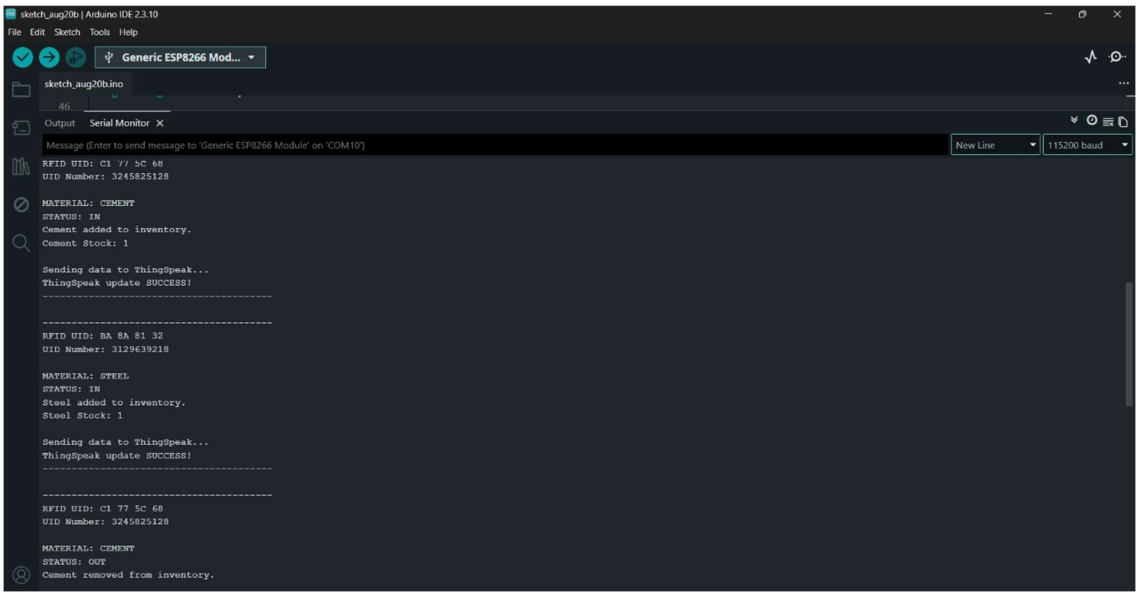
```
// 1 = IN
// 2 = OUT
if (material == "Cement")
{
  if (cementStock > 0)
  {
    ThingSpeak.setField(5, 1);
  }
  else
  {
    ThingSpeak.setField(5, 2);
  }
}
else
{
  if (steelStock > 0)
  {
    ThingSpeak.setField(5, 1);
  }
  else
  {
    ThingSpeak.setField(5, 2);
  }
}

// Write data
```

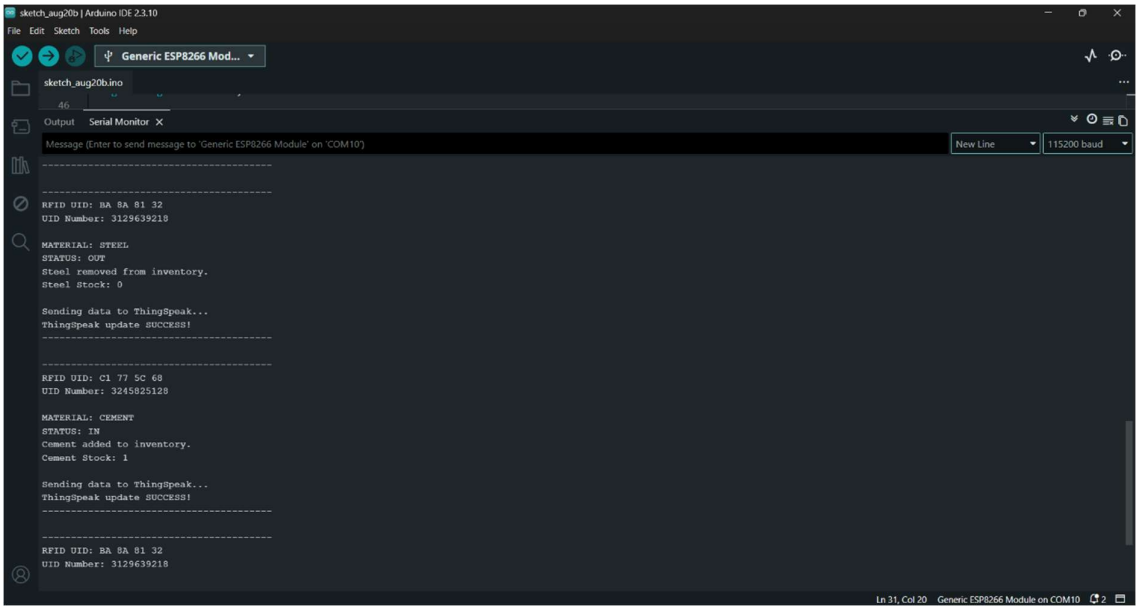
```
  int response = ThingSpeak.writeFields(CHANNEL_ID, WRITE_API_KEY);

  if (response == 200)
  {
    Serial.println("ThingSpeak update SUCCESS!");
  }
  else
  {
    Serial.print("ThingSpeak update FAILED.");
    Serial.print(" HTTP Error: ");
    Serial.println(response);
  }
}

// ============================================================
// WiFi Connection
// ============================================================
void connectWiFi()
{
  Serial.println();
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  int attempts = 0;
```

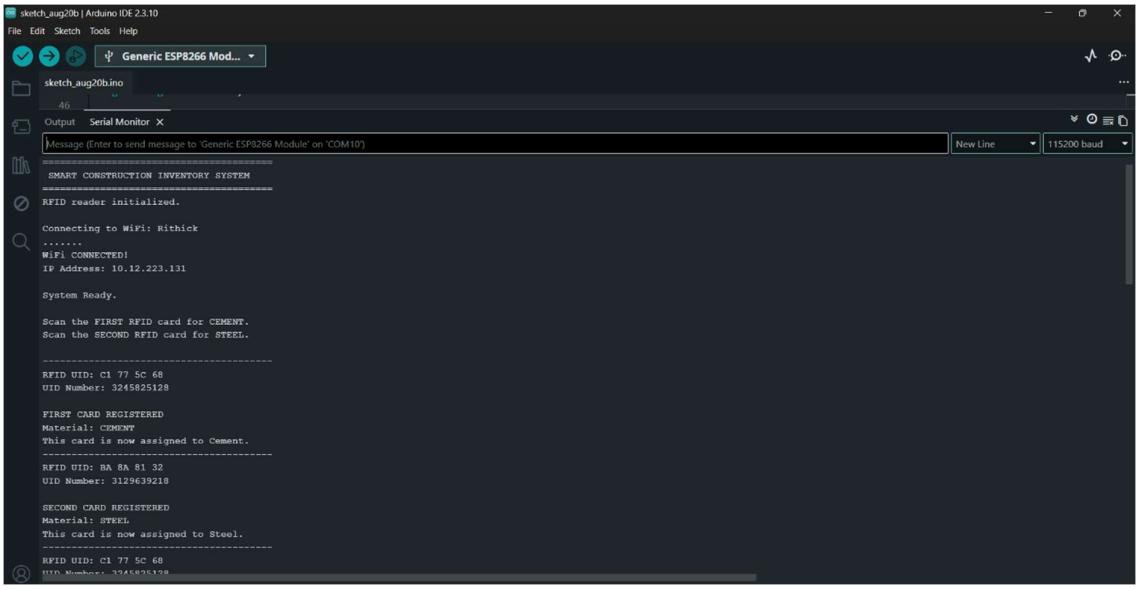
```
  while (WiFi.status() != WL_CONNECTED && attempts < 30)
  {
    delay(500);
    Serial.print(".");

    attempts++;
  }

  Serial.println();

  if (WiFi.status() == WL_CONNECTED)
  {
    Serial.println("WiFi CONNECTED!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
  }
  else
  {
    Serial.println("WiFi CONNECTION FAILED!");
  }
}

// ========================================================
// Success Buzzer
// ========================================================
```

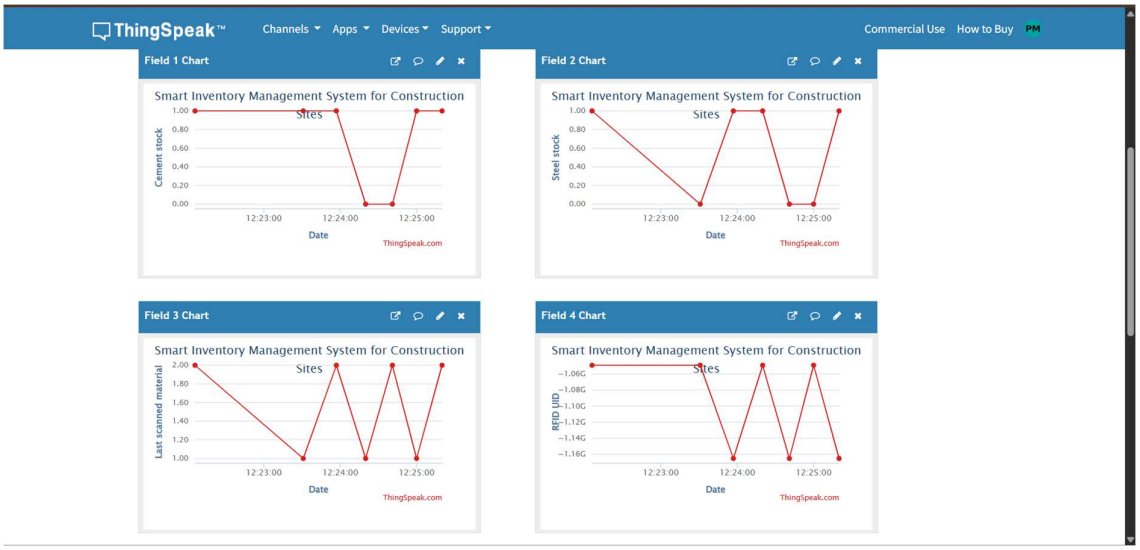
```
void beepSuccess()
{
  digitalWrite(BUZZER_PIN, HIGH);
  delay(150);

  digitalWrite(BUZZER_PIN, LOW);
  delay(100);

  digitalWrite(BUZZER_PIN, HIGH);
  delay(150);

  digitalWrite(BUZZER_PIN, LOW);
}

// ============================================================
// Error Buzzer
// ============================================================
void beepError()
{
  digitalWrite(BUZZER_PIN, HIGH);
  delay(700);
  digitalWrite(BUZZER_PIN, LOW);
}
```

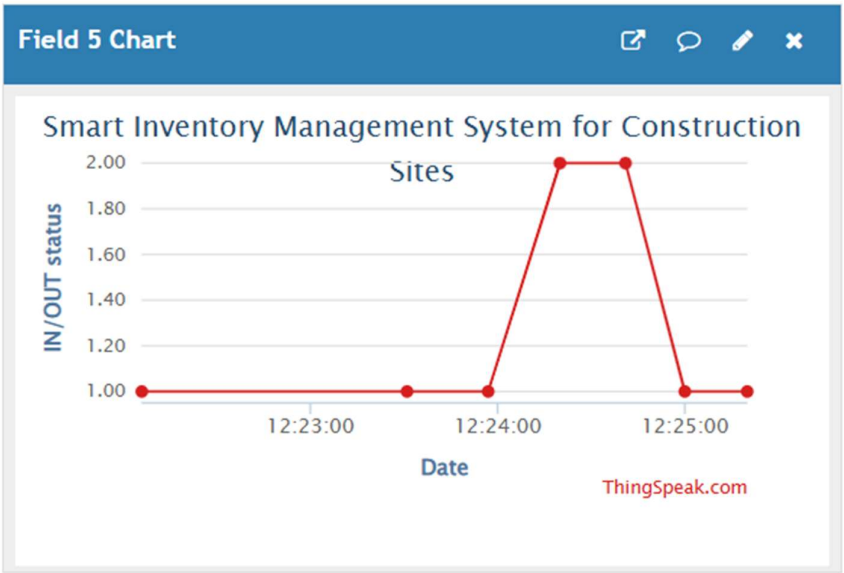
Coding Output:






sketch_aug20b | Arduino IDE 2.3.10
File Edit Sketch Tools Help
Generic ESP8266 Mod...
sketch_aug20b.ino
Output Serial Monitor
Message (Enter to send message to 'Generic ESP8266 Module' on 'COM10')
New Line
115200 baud
SMART CONSTRUCTION INVENTORY SYSTEM
RFID reader initialized.
Connecting to WiFi: Rithick
.....
WiFi CONNECTED!
IP Address: 10.12.223.131
System Ready.
Scan the FIRST RFID card for CEMENT.
Scan the SECOND RFID card for STEEL.
RFID UID: C1 77 5C 68
UID Number: 3245825128
FIRST CARD REGISTERED
Material: CEMENT
This card is now assigned to Cement.
RFID UID: 8A 8A 81 32
UID Number: 3129639218
SECOND CARD REGISTERED
Material: STEEL
This card is now assigned to Steel.
RFID UID: C1 77 5C 68


ThingSpeak™
Channels
Apps
Devices
Support
Commercial Use
How to Buy
PM
Field 1 Chart
Smart Inventory Management System for Construction
Sites
Cement stock
Date
ThingSpeak.com
Field 2 Chart
Smart Inventory Management System for Construction
Sites
Steel stock
Date
ThingSpeak.com
Field 3 Chart
Smart Inventory Management System for Construction
Sites
Last scanned material
Date
ThingSpeak.com
Field 4 Chart
Smart Inventory Management System for Construction
Sites
RFID UID
Date
ThingSpeak.com



System Working

The Smart Inventory Management System uses RFID technology to identify and track construction materials. The ESP8266 Node MCU acts as the main controller, while the MFRC522 RFID reader detects the unique identification number of each RFID tag or card attached to the materials.

When the system is powered ON, the ESP8266 initializes the RFID reader and continuously waits for an RFID tag. When a tagged material is brought near the MFRC522 reader, the reader detects the tag and sends its unique ID to the ESP8266 through SPI communication. The Node MCU compares the received ID with the predefined inventory information and identifies the corresponding material.

After successful identification, the buzzer produces an alert sound to indicate that the material has been detected. The material movement can then be recorded as an inventory transaction, such as material issued, returned, or transferred. The ESP8266's Wi-Fi capability can also be used in future development to send inventory information to a cloud database or monitoring dashboard.

Thus, the system provides automated material identification, movement tracking, and improved inventory management, reducing manual errors and helping construction-site personnel utilize resources more efficiently.

Bill of Materials

S. No	Component	Quantity
1	ESP8266 Node MCU	1
2	MFRC522 RFID Reader	1
3	RFID Tags/Cards	3
4	Buzzer	1
5	Breadboard	1
6	Jumper Wires	1 Set

Prototype Implementation

The Smart Inventory Management System for Construction Sites is implemented using an ESP8266 Node MCU, MFRC522 RFID reader, three RFID tags/cards, buzzer, breadboard, and jumper wires. The prototype is designed to demonstrate automatic identification and basic tracking of construction materials using RFID technology.

First, the ESP8266 Node MCU is placed on the breadboard and connected to the MFRC522 RFID reader using the SPI communication interface. The MFRC522 is supplied with 3.3 V, and its SDA/SS, SCK, MOSI, MISO, and RST pins are connected to the appropriate Node MCU GPIO pins. The buzzer is connected to a digital GPIO pin of the Node MCU to provide an audible indication when a tag is successfully detected.

Three RFID tags/cards are assigned to different construction materials. For example, Tag 1 can represent a construction tool, Tag 2 an electrical material, and Tag 3 a plumbing material. Their unique RFID IDs are programmed into the ESP8266 software along with the corresponding material information.

During implementation, the Node MCU continuously checks the MFRC522 reader for nearby RFID tags. When a tag is detected, its unique ID is read and compared with the registered inventory IDs. If a valid ID is found, the corresponding material is identified and the buzzer is activated. The detected material can then be treated as an inventory movement transaction.

The prototype demonstrates the basic operation of RFID-based material identification and inventory tracking. The ESP8266's built-in Wi-Fi capability also provides a foundation for future integration with a cloud database or web dashboard for remote construction-site inventory monitoring.

Prototype Working

The prototype works by using RFID technology to identify and monitor construction materials. The ESP8266 Node MCU acts as the main controller, while the MFRC522 RFID reader detects the unique ID of each RFID tag or card.

When the system is powered ON, the Node MCU initializes the RFID reader and continuously scans for nearby RFID tags. Each RFID tag is assigned to a particular construction material. When a tag is placed near the MFRC522 reader, the reader detects its unique ID and sends the information to the ESP8266 through SPI communication.

The ESP8266 compares the detected ID with the predefined inventory information. If the tag is registered, the corresponding material is identified and the buzzer produces an alert sound to indicate successful detection. The material movement can then be recorded as an inventory transaction, such as issued, returned, or transferred.

For example, three RFID cards can represent three different inventory items. When RFID Tag 1 is scanned, the system identifies the material assigned to Tag 1 and activates the buzzer. The same process is repeated for Tags 2 and 3.

Thus, the prototype demonstrates automatic material identification, RFID-based inventory tracking, and movement monitoring, providing a simple foundation for a larger construction-site inventory management system.

Expected Prototype Output

The Smart Inventory Management System is expected to successfully identify and monitor construction materials using RFID technology. When an RFID-tagged material is brought near the MFRC522 reader, the system reads its unique ID and the ESP8266 identifies the corresponding inventory item.

The expected prototype outputs are:

- **RFID Tag Detected:** The system detects the RFID card/tag placed near the reader.
- **Material Identification:** The ESP8266 identifies the material associated with the detected RFID ID.
- **Buzzer Alert:** The buzzer produces a sound to indicate successful RFID detection.
- **Inventory Movement:** The detected material can be recorded as issued, returned, or transferred.
- **Invalid Tag Detection:** Unregistered RFID tags can be identified as unauthorized or unknown items.
- **Multiple Material Tracking:** Three RFID tags can be assigned to three different construction materials.
- **Future IoT Monitoring:** The ESP8266 can transmit inventory information through Wi-Fi to a cloud platform or web dashboard.

Prototype Demonstration

- The prototype demonstration begins by powering the ESP8266 Node MCU and allowing it to initialize the MFRC522 RFID reader. The RFID reader is positioned at the material identification point, while the three RFID tags/cards are assigned to different construction materials.
- During the demonstration, RFID Tag 1 is brought close to the MFRC522 reader. The reader detects the tag and sends its unique identification number to the ESP8266. The Node MCU verifies the ID and identifies the corresponding inventory item. The buzzer produces an alert sound to indicate successful detection.

- The same procedure is repeated with RFID Tag 2 and RFID Tag 3. Each tag is identified according to its registered inventory information. The demonstration can also show an unregistered RFID tag to verify that the system can distinguish between valid and unknown inventory items.
- The prototype demonstrates the complete process of RFID detection → material identification → buzzer alert → inventory movement recording. This shows how the system can reduce manual inventory work and provide efficient material tracking for construction-site management.

Results and Performance Analysis

The developed Smart Inventory Management System for Construction Sites successfully demonstrates RFID-based identification and tracking of construction materials. The MFRC522 RFID reader detects the unique identification number of each RFID tag, while the ESP8266 Node MCU processes the received data and identifies the corresponding inventory item.

The buzzer provides an immediate audible indication when a registered RFID tag is detected. The prototype successfully demonstrates the basic concept of recording material movement and distinguishing between registered and unregistered RFID tags.

The system provides the following results:

- Successful detection of RFID tags/cards.
- Unique identification of registered construction materials.
- Immediate buzzer indication after successful detection.
- Basic tracking of material movement.
- Reduced dependence on manual inventory identification.
- Capability for future Wi-Fi and cloud-based inventory monitoring.
- Simple and low-cost implementation suitable for prototype development.

Performance Analysis:

Performance Parameter	Observed/Expected Result
RFID Tag Detection	Successfully detects registered RFID tags within the reader's operating range
Material Identification	Accurate identification based on unique RFID ID
Response Time	Fast response after tag detection
Buzzer Response	Activates immediately after successful tag recognition
Inventory Tracking	Supports basic material movement monitoring
Multiple Tags	Supports multiple registered RFID tags
Reliability	Suitable for continuous prototype-level operation
Connectivity	ESP8266 provides Wi-Fi capability for future cloud integration
Scalability	Additional RFID tags can be registered as inventory increases
Manual Effort	Reduces manual material identification and recording